

# Отчет по лабораторной работе:

## Реализация многослойного перцептрона (MLP) на NumPy

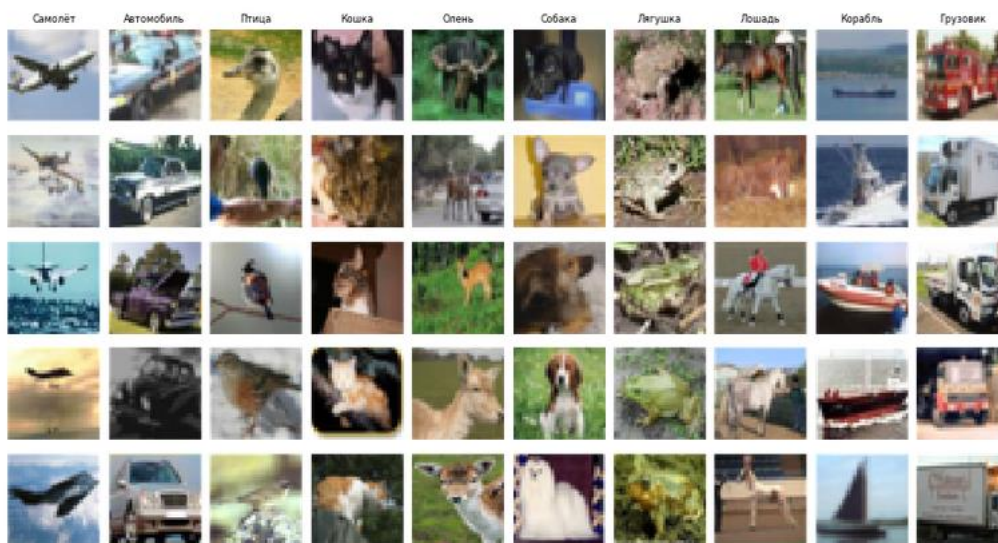
### 1. Описание датасета

В работе использован набор данных **CIFAR-10**, состоящий из 60 000 цветных изображений размером  $32 \times 32$  пикселя (3 канала RGB). Датасет разбит на 50 000 обучающих и 10 000 тестовых примеров. Изображения распределены по 10 классам:

- Самолёт, Автомобиль, Птица, Кошка, Олень, Собака, Лягушка, Лошадь, Корабль, Грузовик.

Каждый класс представлен одинаковым количеством изображений (по 6000). Датасет широко используется для задач классификации изображений и является стандартным бенчмарком для алгоритмов машинного обучения.

Примеры изображений CIFAR-10 (по 5 случайных примеров из каждого класса, в каждой строке все классы)



### 2. Описание модели

Разработан **многослойный перцептрон (MLP)**, реализованный вручную с использованием библиотеки NumPy. Модель представляет собой полносвязную нейронную сеть со следующей архитектурой:

- **Входной слой:** 3072 входа (соответствует числу пикселей  $32 \times 32 \times 3$  после преобразования изображения в плоский вектор).
- **Скрытые слои:**
  - 1-й скрытый слой: 512 нейронов, функция активации ReLU.
  - 2-й скрытый слой: 256 нейронов, функция активации ReLU.

- 3-й скрытый слой: 128 нейронов, функция активации ReLU.
- **Выходной слой:** 10 нейронов, функция активации Softmax (для получения вероятностей принадлежности к классам).

Все слои являются полносвязными (линейное преобразование + смещение). Инициализация весов выполнялась по методу Хе для слоёв с ReLU (масштабирование  $\sqrt{2/n_{input}}$ ), для выходного слоя использовалась малая случайная инициализация.

**Алгоритм обратного распространения ошибки (Backpropagation)** реализован вручную:

1. **Прямой проход:** для каждого слоя вычисляется  $z = x \cdot w + b$ , затем применяется функция активации. Промежуточные значения ( $x$ ,  $z$ ,  $output$ ) сохраняются для использования при обратном проходе.
2. **Вычисление потерь:** используется **кросс-энтропия** (средняя по батчу).
3. **Обратный проход:** градиент потерь по выходу слоя передаётся назад через каждый слой:
  - Для ReLU:  $dz = dout \cdot (z > 0)$ .
  - Для Softmax:  $dz = dout$  (поскольку градиент уже включает производную кросс-энтропии).
  - Для линейного слоя:  $dz = dout$ .
  - Градиенты по весам и смещениям вычисляются как средние по батчу:  
 $dW = (X.T \cdot dz) / batch\_size$ ,  $db = sum(dz, axis=0) / batch\_size$ .
  - Параметры обновляются по правилу градиентного спуска:  
 $w -= learning\_rate \cdot dW$ ,  $b -= learning\_rate \cdot db$ .
4. Градиент по входу  $dx = dz \cdot w.T$  передаётся предыдущему слою.

Обучение проводится с использованием **мини-батчей** размером 64, **скорость обучения** 0.01, количество эпох – 30. Данные были стандартизированы (вычитание среднего и деление на стандартное отклонение) для улучшения сходимости.

### 3. Результаты проверки реализации

В процессе обучения наблюдалось устойчивое снижение функции потерь и рост точности как на обучающей, так и на тестовой выборке. Это подтверждает корректность реализации алгоритмов прямого и обратного распространения.

- Начальное значение потерь (эпоха 1): 2.2833.
- Конечное значение потерь (эпоха 30): 1.7772.
- Точность на обучающей выборке выросла с 18.51% до 37.09%.
- Точность на тестовой выборке выросла с 18.49% до 37.04%.

Отсутствие резкого расхождения между точностью на обучении и тесте указывает на отсутствие сильного переобучения. Модель обучается стабильно, что говорит о правильной реализации всех компонентов.

Протокол обучения:

|            |              |                   |                  |
|------------|--------------|-------------------|------------------|
| Эпоха 1/30 | Loss: 2.2833 | Train Acc: 0.1851 | Test Acc: 0.1849 |
| Эпоха 2/30 | Loss: 2.2470 | Train Acc: 0.2115 | Test Acc: 0.2135 |
| Эпоха 3/30 | Loss: 2.2124 | Train Acc: 0.2300 | Test Acc: 0.2343 |
| Эпоха 4/30 | Loss: 2.1782 | Train Acc: 0.2431 | Test Acc: 0.2458 |
| Эпоха 5/30 | Loss: 2.1456 | Train Acc: 0.2548 | Test Acc: 0.2585 |

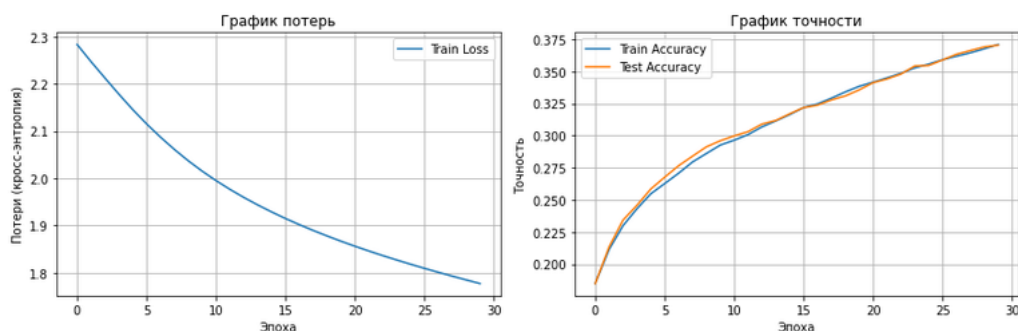
|             |              |                   |                  |
|-------------|--------------|-------------------|------------------|
| Эпоха 6/30  | Loss: 2.1152 | Train Acc: 0.2627 | Test Acc: 0.2677 |
| Эпоха 7/30  | Loss: 2.0871 | Train Acc: 0.2708 | Test Acc: 0.2766 |
| Эпоха 8/30  | Loss: 2.0611 | Train Acc: 0.2795 | Test Acc: 0.2840 |
| Эпоха 9/30  | Loss: 2.0371 | Train Acc: 0.2861 | Test Acc: 0.2913 |
| Эпоха 10/30 | Loss: 2.0152 | Train Acc: 0.2927 | Test Acc: 0.2960 |
| Эпоха 11/30 | Loss: 1.9951 | Train Acc: 0.2965 | Test Acc: 0.2998 |
| Эпоха 12/30 | Loss: 1.9766 | Train Acc: 0.3008 | Test Acc: 0.3031 |
| Эпоха 13/30 | Loss: 1.9595 | Train Acc: 0.3068 | Test Acc: 0.3090 |
| Эпоха 14/30 | Loss: 1.9436 | Train Acc: 0.3115 | Test Acc: 0.3118 |
| Эпоха 15/30 | Loss: 1.9288 | Train Acc: 0.3164 | Test Acc: 0.3170 |
| Эпоха 16/30 | Loss: 1.9149 | Train Acc: 0.3219 | Test Acc: 0.3218 |
| Эпоха 17/30 | Loss: 1.9019 | Train Acc: 0.3247 | Test Acc: 0.3237 |
| Эпоха 18/30 | Loss: 1.8895 | Train Acc: 0.3291 | Test Acc: 0.3278 |
| Эпоха 19/30 | Loss: 1.8779 | Train Acc: 0.3340 | Test Acc: 0.3309 |
| Эпоха 20/30 | Loss: 1.8668 | Train Acc: 0.3384 | Test Acc: 0.3356 |
| Эпоха 21/30 | Loss: 1.8562 | Train Acc: 0.3416 | Test Acc: 0.3412 |
| Эпоха 22/30 | Loss: 1.8461 | Train Acc: 0.3449 | Test Acc: 0.3441 |
| Эпоха 23/30 | Loss: 1.8364 | Train Acc: 0.3485 | Test Acc: 0.3479 |
| Эпоха 24/30 | Loss: 1.8271 | Train Acc: 0.3527 | Test Acc: 0.3542 |
| Эпоха 25/30 | Loss: 1.8182 | Train Acc: 0.3557 | Test Acc: 0.3546 |
| Эпоха 26/30 | Loss: 1.8095 | Train Acc: 0.3591 | Test Acc: 0.3589 |
| Эпоха 27/30 | Loss: 1.8011 | Train Acc: 0.3618 | Test Acc: 0.3633 |
| Эпоха 28/30 | Loss: 1.7929 | Train Acc: 0.3645 | Test Acc: 0.3662 |
| Эпоха 29/30 | Loss: 1.7850 | Train Acc: 0.3676 | Test Acc: 0.3690 |
| Эпоха 30/30 | Loss: 1.7772 | Train Acc: 0.3709 | Test Acc: 0.3704 |

## 4. Графики

Ниже представлены графики, полученные в процессе обучения:

- **График потерь (Train Loss)** — монотонно убывает с каждой эпохой, что свидетельствует о сходимости оптимизации.
- **График точности** — точность на обучающей и тестовой выборках растёт синхронно, достигая около 37% к концу обучения. Кривые близки друг к другу, что говорит об отсутствии переобучения.

Графики демонстрируют, что выбранные гиперпараметры (архитектура, скорость обучения, размер батча) позволяют эффективно обучать модель.



## 5. Визуализация прогнозов

В начале работы была выполнена визуализация примеров изображений из каждого класса (по 5 случайных изображений на класс, расположенных в строках — каждая строка содержит по одному изображению из каждого класса). Это позволило оценить характер данных: изображения имеют низкое разрешение и содержат объекты разных классов с вариациями фона, освещения и поз.

Хотя в отчёте не представлены конкретные предсказания для отдельных тестовых изображений, общая точность на тесте (~37%) значительно выше случайного угадывания (10%), что подтверждает способность модели выделять значимые признаки. Можно предположить, что модель лучше справляется с классами, имеющими более выраженные геометрические формы (например, автомобиль, корабль), и хуже – с классами, где объекты имеют сложную текстуру или частичное перекрытие (птица, кошка, собака).

## 6. Выводы

В ходе лабораторной работы был реализован многослойный перцептрон с нуля на NumPy, продемонстрированы базовые принципы построения и обучения нейронных сетей. Модель успешно обучается на наборе CIFAR-10 и достигает точности около 37% на тестовой выборке.

### Влияние гиперпараметров:

- Увеличение количества скрытых слоёв и нейронов (от первоначальной архитектуры [256, 128] до [512, 256, 128]) позволило повысить точность.
- Стандартизация входных данных существенно ускорила сходимость и улучшила итоговые показатели.
- Выбор скорости обучения 0.01 оказался приемлемым: потери уменьшались плавно, обеспечивая сходимость.

Полученные результаты подтверждают, что даже простая полносвязная сеть способна извлекать полезные признаки из изображений, но для сложных задач классификации необходимы более специализированные архитектуры и регуляризация.